DGT2823 - Tecnologias para Desenvolvimento de Soluções de Big Data

Microatividades (1 a 5) e Trabalho Prático

Contextualização

Como Analista de Dados, você recebeu, em um novo projeto, um conjunto de dados. Sua principal tarefa é tratar os dados desse conjunto a fim de que possam ser utilizados para a descoberta de conhecimento através de sua posterior análise e interpretação. Para tal tarefa, são utilizadas a linguagem **Python** e a biblioteca **Pandas**.

Objetivos da Prática

- Ler um arquivo CSV usando a biblioteca Pandas (`read_csv`);
- Criar um subconjunto de dados a partir de um conjunto existente;
- Configurar o número máximo de linhas exibidas (`max_rows = 9999`) e usar `to_string()` ;
- Exibir as primeiras e últimas N linhas (`head` e `tail`);
- Extrair informações gerais sobre colunas, linhas, tipos de dados, valores nulos e uso de memória (`info`);
- Executar o pipeline completo de limpeza e transformação de dados (tratamento de nulos em `Calories` e padronização/conversão de datas em `Date`).

Preparação do Ambiente: Instalação Automática de Dependências

Objetivo: Garantir que as bibliotecas `pandas` e `numpy` estejam instaladas diretamente no ambiente (kernel Python) em que este Jupyter Notebook está sendo executado.

- O comando mágico `%pip install pandas numpy` (ou `!pip install pandas numpy` via subshell) verifica se os pacotes já estão presentes no interpretador ativo e, caso não estejam, realiza o download e a instalação automaticamente antes da execução das microatividades e do trabalho prático.

In [1]:
```
# Instalação automática das bibliotecas necessárias no kernel ativo do Jupyter
%pip install pandas numpy
```

```
Requirement already satisfied: pandas in .\.venv\Lib\site-packages (3.1.0)
Requirement already satisfied: numpy in .\.venv\Lib\site-packages (2.5.4)
Requirement already satisfied: python-dateutil>=2.9.0 in .\.venv\Lib\site-packages
(from pandas) (3.0.0)
Requirement already satisfied: tzdata in .\.venv\Lib\site-packages (from pandas) (20
26.6)
Requirement already satisfied: six>=1.6 in .\.venv\Lib\site-packages (from python-da
teutil>=2.9.0->pandas) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

Parte 1: Microatividades Exploratórias (1 a 5)

Microatividade 1: Leitura de Arquivo CSV com Pandas

Objetivo: Importar a biblioteca `pandas`, ler o conteúdo do arquivo CSV passando o separador de colunas (`sep=';'`), a engine (`engine='python'`) e a codificação (`encoding='utf-8'`), atribuir os dados a uma variável e exibi-los em tela.

In [2]:
```python
import pandas as pd
import numpy as np

# Caminho para o arquivo CSV do conjunto de dados
caminho_csv = 'conjunto_de_dados_trabalho_pratico_contextualizacao_atividade_pico_w

# Leitura do CSV com os parâmetros solicitados na Microatividade 1
df_original = pd.read_csv(
    caminho_csv,
    sep=';',
    engine='python',
    encoding='utf-8'
)

# Exibição dos dados lidos
print(df_original)
```

	ID	Duration	Date	Pulse	Maxpulse	Calories
0	0	60	'2020/12/01'	110	130	4091.0
1	1	60	'2020/12/02'	117	145	4790.0
2	2	60	'2020/12/03'	103	135	3400.0
3	3	45	'2020/12/04'	109	175	2824.0
4	4	45	'2020/12/05'	117	148	4060.0
5	5	60	'2020/12/06'	102	127	3000.0
6	6	60	'2020/12/07'	110	136	3740.0
7	7	450	'2020/12/08'	104	134	2533.0
8	8	30	'2020/12/09'	109	133	1951.0
9	9	60	'2020/12/10'	98	124	2690.0
10	10	60	'2020/12/11'	103	147	3293.0
11	11	60	'2020/12/12'	100	120	2507.0
12	12	60	'2020/12/12'	100	120	2507.0
13	13	60	'2020/12/13'	106	128	3453.0
14	14	60	'2020/12/14'	104	132	3793.0
15	15	60	'2020/12/15'	98	123	2750.0
16	16	60	'2020/12/16'	98	120	2152.0
17	17	60	'2020/12/17'	100	120	3000.0
18	18	45	'2020/12/18'	90	112	NaN
19	19	60	'2020/12/19'	103	123	3230.0
20	20	45	'2020/12/20'	97	125	2430.0
21	21	60	'2020/12/21'	108	131	3642.0
22	22	45	NaN	100	119	2820.0
23	23	60	'2020/12/23'	130	101	3000.0
24	24	45	'2020/12/24'	105	132	2460.0
25	25	60	'2020/12/25'	102	126	3345.0
26	26	60	20201226	100	120	2500.0
27	27	60	'2020/12/27'	92	118	2410.0
28	28	60	'2020/12/28'	103	132	NaN
29	29	60	'2020/12/29'	100	132	2800.0
30	30	60	'2020/12/30'	102	129	3803.0
31	31	60	'2020/12/31'	92	115	2430.0

Microatividade 2: Criação de Subconjunto de Dados

Objetivo: Criar uma nova variável contendo um subconjunto de dados com apenas 3 colunas do conjunto original (`ID` , `Date` e `Calories`) e exibir seu conteúdo em tela.

In [3]:
```
# Criação do subconjunto com 3 colunas
df_subconjunto = df_original[['ID', 'Date', 'Calories']]

# Exibição do subconjunto de dados
print(df_subconjunto)
```

```
    ID          Date  Calories
0    0  '2020/12/01'    4091.0
1    1  '2020/12/02'    4790.0
2    2  '2020/12/03'    3400.0
3    3  '2020/12/04'    2824.0
4    4  '2020/12/05'    4060.0
5    5  '2020/12/06'    3000.0
6    6  '2020/12/07'    3740.0
7    7  '2020/12/08'    2533.0
8    8  '2020/12/09'    1951.0
9    9  '2020/12/10'    2690.0
10  10  '2020/12/11'    3293.0
11  11  '2020/12/12'    2507.0
12  12  '2020/12/12'    2507.0
13  13  '2020/12/13'    3453.0
14  14  '2020/12/14'    3793.0
15  15  '2020/12/15'    2750.0
16  16  '2020/12/16'    2152.0
17  17  '2020/12/17'    3000.0
18  18  '2020/12/18'       NaN
19  19  '2020/12/19'    3230.0
20  20  '2020/12/20'    2430.0
21  21  '2020/12/21'    3642.0
22  22           NaN    2820.0
23  23  '2020/12/23'    3000.0
24  24  '2020/12/24'    2460.0
25  25  '2020/12/25'    3345.0
26  26      20201226    2500.0
27  27  '2020/12/27'    2410.0
28  28  '2020/12/28'       NaN
29  29  '2020/12/29'    2800.0
30  30  '2020/12/30'    3803.0
31  31  '2020/12/31'    2430.0
```

Microatividade 3: Configuração de `max_rows` e Exibição com `to_string()`

Objetivo: Definir um novo valor para a propriedade `display.max_rows` (`9999`) nas opções de configuração do Pandas e imprimir o conjunto de dados original utilizando o método `.to_string()` .

In [4]:
```python
# Configuração do número máximo de linhas para 9999
pd.options.display.max_rows = 9999

# Exibição completa do conjunto de dados original como string
print(df_original.to_string())
```

	ID	Duration	Date	Pulse	Maxpulse	Calories
0	0	60	'2020/12/01'	110	130	4091.0
1	1	60	'2020/12/02'	117	145	4790.0
2	2	60	'2020/12/03'	103	135	3400.0
3	3	45	'2020/12/04'	109	175	2824.0
4	4	45	'2020/12/05'	117	148	4060.0
5	5	60	'2020/12/06'	102	127	3000.0
6	6	60	'2020/12/07'	110	136	3740.0
7	7	450	'2020/12/08'	104	134	2533.0
8	8	30	'2020/12/09'	109	133	1951.0
9	9	60	'2020/12/10'	98	124	2690.0
10	10	60	'2020/12/11'	103	147	3293.0
11	11	60	'2020/12/12'	100	120	2507.0
12	12	60	'2020/12/12'	100	120	2507.0
13	13	60	'2020/12/13'	106	128	3453.0
14	14	60	'2020/12/14'	104	132	3793.0
15	15	60	'2020/12/15'	98	123	2750.0
16	16	60	'2020/12/16'	98	120	2152.0
17	17	60	'2020/12/17'	100	120	3000.0
18	18	45	'2020/12/18'	90	112	NaN
19	19	60	'2020/12/19'	103	123	3230.0
20	20	45	'2020/12/20'	97	125	2430.0
21	21	60	'2020/12/21'	108	131	3642.0
22	22	45	NaN	100	119	2820.0
23	23	60	'2020/12/23'	130	101	3000.0
24	24	45	'2020/12/24'	105	132	2460.0
25	25	60	'2020/12/25'	102	126	3345.0
26	26	60	20201226	100	120	2500.0
27	27	60	'2020/12/27'	92	118	2410.0
28	28	60	'2020/12/28'	103	132	NaN
29	29	60	'2020/12/29'	100	132	2800.0
30	30	60	'2020/12/30'	102	129	3803.0
31	31	60	'2020/12/31'	92	115	2430.0

Microatividade 4: Exibição das Primeiras e Últimas 10 Linhas

Objetivo: Imprimir na tela apenas as primeiras 10 linhas (`.head(10)`) e as últimas 10 linhas (`.tail(10)`) do conjunto de dados original.

In [5]:
```
print('--- Primeiras 10 Linhas ---')
print(df_original.head(10))

print('\n--- Últimas 10 Linhas ---')
print(df_original.tail(10))
```

```
--- Primeiras 10 Linhas ---
   ID  Duration          Date  Pulse  Maxpulse  Calories
0   0        60  '2020/12/01'    110       130    4091.0
1   1        60  '2020/12/02'    117       145    4790.0
2   2        60  '2020/12/03'    103       135    3400.0
3   3        45  '2020/12/04'    109       175    2824.0
4   4        45  '2020/12/05'    117       148    4060.0
5   5        60  '2020/12/06'    102       127    3000.0
6   6        60  '2020/12/07'    110       136    3740.0
7   7       450  '2020/12/08'    104       134    2533.0
8   8        30  '2020/12/09'    109       133    1951.0
9   9        60  '2020/12/10'     98       124    2690.0

--- Últimas 10 Linhas ---
    ID  Duration          Date  Pulse  Maxpulse  Calories
22  22        45           NaN    100       119    2820.0
23  23        60  '2020/12/23'    130       101    3000.0
24  24        45  '2020/12/24'    105       132    2460.0
25  25        60  '2020/12/25'    102       126    3345.0
26  26        60      20201226    100       120    2500.0
27  27        60  '2020/12/27'     92       118    2410.0
28  28        60  '2020/12/28'    103       132       NaN
29  29        60  '2020/12/29'    100       132    2800.0
30  30        60  '2020/12/30'    102       129    3803.0
31  31        60  '2020/12/31'     92       115    2430.0
```

Microatividade 5: Informações Gerais sobre o Conjunto de Dados

Objetivo: Utilizar o método `.info()` para inspecionar a estrutura do DataFrame e responder às questões sobre total de linhas, total de colunas, dados nulos, tipos de dados e consumo de memória.

In [6]:
```
# Exibição das informações gerais sobre o conjunto de dados
df_original.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 32 entries, 0 to 31
Data columns (total 6 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   ID        32 non-null     int64
 1   Duration  32 non-null     int64
 2   Date      31 non-null     str
 3   Pulse     32 non-null     int64
 4   Maxpulse  32 non-null     int64
 5   Calories  30 non-null     float64
dtypes: float64(1), int64(4), str(1)
memory usage: 1.6 KB
```

Respostas da Microatividade 5 (extraídas de `df_original.info()`):

1. **Total de linhas:** `32` (`RangeIndex: 32 entries, 0 to 31`).

2. **Total de colunas:** `6` (`ID` , `Duration` , `Date` , `Pulse` , `Maxpulse` , `Calories`).
3. **Quantidade de dados nulos:** `3` valores nulos no total — sendo `1` na coluna `Date` (`31 non-null` , linha 22) e `2` na coluna `Calories` (`30 non-null` , linhas 18 e 28).
4. **Tipo de dado de cada coluna (`Dtype`):**
   - `ID` , `Duration` , `Pulse` , `Maxpulse` : `int64`
   - `Date` : `object` (texto/string)
   - `Calories` : `float64`
5. **Quantidade de memória utilizada:** `1.6+ KB` .

Parte 2: Trabalho Prático — Tratamento e Limpeza de Dados (Passos 1 a 13)

Passos 1 a 5: Importação e Verificação Inicial do Conjunto de Dados

Objetivo: Ler o arquivo CSV fornecido (composto pelas colunas `ID; Duration; Date; Pulse; Maxpulse; Calories`), atribuir a uma variável e verificar a importação imprimindo as informações gerais e as primeiras/últimas N linhas.

In [7]:
```
# Passos 1 a 4: Leitura do CSV
df_bruto = pd.read_csv(
    caminho_csv,
    sep=';',
    engine='python',
    encoding='utf-8'
)

# Passo 5a: Informações gerais
print('--- Passo 5a: Informações Gerais ---')
df_bruto.info()

# Passo 5b: Primeiras e últimas 5 linhas
print('\n--- Passo 5b: Primeiras 5 Linhas ---')
print(df_bruto.head(5))

print('\n--- Passo 5b: Últimas 5 Linhas ---')
print(df_bruto.tail(5))
```

```
--- Passo 5a: Informações Gerais ---
<class 'pandas.DataFrame'>
RangeIndex: 32 entries, 0 to 31
Data columns (total 6 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   ID        32 non-null     int64
 1   Duration  32 non-null     int64
 2   Date      31 non-null     str
 3   Pulse     32 non-null     int64
 4   Maxpulse  32 non-null     int64
 5   Calories  30 non-null     float64
dtypes: float64(1), int64(4), str(1)
memory usage: 1.6 KB

--- Passo 5b: Primeiras 5 Linhas ---
   ID  Duration          Date  Pulse  Maxpulse  Calories
0   0        60  '2020/12/01'    110       130    4091.0
1   1        60  '2020/12/02'    117       145    4790.0
2   2        60  '2020/12/03'    103       135    3400.0
3   3        45  '2020/12/04'    109       175    2824.0
4   4        45  '2020/12/05'    117       148    4060.0

--- Passo 5b: Últimas 5 Linhas ---
    ID  Duration          Date  Pulse  Maxpulse  Calories
27  27        60  '2020/12/27'     92       118    2410.0
28  28        60  '2020/12/28'    103       132       NaN
29  29        60  '2020/12/29'    100       132    2800.0
30  30        60  '2020/12/30'    102       129    3803.0
31  31        60  '2020/12/31'     92       115    2430.0
```

Passos 6 e 7: Cópia do Conjunto de Dados e Tratamento de Nulos em `Calories`

Objetivo: Criar uma cópia independente do DataFrame original (`.copy()`), substituir todos os valores nulos (`NaN`) da coluna `Calories` (linhas 18 e 28) por `0` utilizando `.fillna(0)` e imprimir o conjunto de dados para conferência.

In [8]:
```python
# Passo 6: Criação de uma cópia do conjunto de dados original
df_tratado = df_bruto.copy()

# Passo 7a: Substituição de valores nulos na coluna 'Calories' por 0
df_tratado['Calories'] = df_tratado['Calories'].fillna(0)

# Passo 7b: Impressão para verificar a substituição nas linhas 18 e 28
print(df_tratado.to_string())
```

	ID	Duration	Date	Pulse	Maxpulse	Calories
0	0	60	'2020/12/01'	110	130	4091.0
1	1	60	'2020/12/02'	117	145	4790.0
2	2	60	'2020/12/03'	103	135	3400.0
3	3	45	'2020/12/04'	109	175	2824.0
4	4	45	'2020/12/05'	117	148	4060.0
5	5	60	'2020/12/06'	102	127	3000.0
6	6	60	'2020/12/07'	110	136	3740.0
7	7	450	'2020/12/08'	104	134	2533.0
8	8	30	'2020/12/09'	109	133	1951.0
9	9	60	'2020/12/10'	98	124	2690.0
10	10	60	'2020/12/11'	103	147	3293.0
11	11	60	'2020/12/12'	100	120	2507.0
12	12	60	'2020/12/12'	100	120	2507.0
13	13	60	'2020/12/13'	106	128	3453.0
14	14	60	'2020/12/14'	104	132	3793.0
15	15	60	'2020/12/15'	98	123	2750.0
16	16	60	'2020/12/16'	98	120	2152.0
17	17	60	'2020/12/17'	100	120	3000.0
18	18	45	'2020/12/18'	90	112	0.0
19	19	60	'2020/12/19'	103	123	3230.0
20	20	45	'2020/12/20'	97	125	2430.0
21	21	60	'2020/12/21'	108	131	3642.0
22	22	45	NaN	100	119	2820.0
23	23	60	'2020/12/23'	130	101	3000.0
24	24	45	'2020/12/24'	105	132	2460.0
25	25	60	'2020/12/25'	102	126	3345.0
26	26	60	20201226	100	120	2500.0
27	27	60	'2020/12/27'	92	118	2410.0
28	28	60	'2020/12/28'	103	132	0.0
29	29	60	'2020/12/29'	100	132	2800.0
30	30	60	'2020/12/30'	102	129	3803.0
31	31	60	'2020/12/31'	92	115	2430.0

Passo 8: Substituição de Nulos em `Date` por `'1900/01/01'` e Primeira Tentativa com `to_datetime`

Objetivo: Substituir os valores nulos da coluna `Date` (linha 22) por `'1900/01/01'`, imprimir o DataFrame e tentar converter a coluna `Date` para o tipo `datetime` usando `pd.to_datetime()`.

> **Nota Pedagógica:** Como as datas das linhas 0 a 21 possuem aspas simples embutidas (ex.: `'2020/12/01'`), o Pandas infere o formato `'%Y/%m/%d'` (com aspas simples) e lança um `ValueError` ao encontrar `'1900/01/01'` (sem aspas simples) na posição 22 — exatamente como descrito no enunciado do Passo 9.

In [9]:
```
# Passo 8a: Substituição dos valores nulos da coluna 'Date' por '1900/01/01'
df_tratado['Date'] = df_tratado['Date'].fillna('1900/01/01')

# Passo 8b: Impressão do conjunto de dados para conferência
```

```
print(df_tratado.to_string())

# Passo 8c: Transformação de 'Date' em datetime (capturando o erro esperado no Pass
print('\n--- Tentativa de conversão com pd.to_datetime (Passo 8c) ---')
try:
    df_tratado['Date'] = pd.to_datetime(df_tratado['Date'])
except ValueError as e:
    print(f'ValueError esperado (Passo 9): {e}')
```

```
    ID  Duration          Date  Pulse  Maxpulse  Calories
0    0        60  '2020/12/01'    110       130    4091.0
1    1        60  '2020/12/02'    117       145    4790.0
2    2        60  '2020/12/03'    103       135    3400.0
3    3        45  '2020/12/04'    109       175    2824.0
4    4        45  '2020/12/05'    117       148    4060.0
5    5        60  '2020/12/06'    102       127    3000.0
6    6        60  '2020/12/07'    110       136    3740.0
7    7       450  '2020/12/08'    104       134    2533.0
8    8        30  '2020/12/09'    109       133    1951.0
9    9        60  '2020/12/10'     98       124    2690.0
10  10        60  '2020/12/11'    103       147    3293.0
11  11        60  '2020/12/12'    100       120    2507.0
12  12        60  '2020/12/12'    100       120    2507.0
13  13        60  '2020/12/13'    106       128    3453.0
14  14        60  '2020/12/14'    104       132    3793.0
15  15        60  '2020/12/15'     98       123    2750.0
16  16        60  '2020/12/16'     98       120    2152.0
17  17        60  '2020/12/17'    100       120    3000.0
18  18        45  '2020/12/18'     90       112       0.0
19  19        60  '2020/12/19'    103       123    3230.0
20  20        45  '2020/12/20'     97       125    2430.0
21  21        60  '2020/12/21'    108       131    3642.0
22  22        45    1900/01/01    100       119    2820.0
23  23        60  '2020/12/23'    130       101    3000.0
24  24        45  '2020/12/24'    105       132    2460.0
25  25        60  '2020/12/25'    102       126    3345.0
26  26        60      20201226    100       120    2500.0
27  27        60  '2020/12/27'     92       118    2410.0
28  28        60  '2020/12/28'    103       132       0.0
29  29        60  '2020/12/29'    100       132    2800.0
30  30        60  '2020/12/30'    102       129    3803.0
31  31        60  '2020/12/31'     92       115    2430.0

--- Tentativa de conversão com pd.to_datetime (Passo 8c) ---
ValueError esperado (Passo 9): time data "1900/01/01" doesn't match format "'%Y/%m/%
d'". You might want to try:
    - passing `format` if your strings have a consistent format;
    - passing `format='ISO8601'` if your strings are all ISO8601 but not necessarily
in exactly the same format;
    - passing `format='mixed'`, and the format will be inferred for each element ind
ividually. You might want to use `dayfirst` alongside this.
```

Passo 9: Substituição de `'1900/01/01'` por `NaN` e Segunda Tentativa com `to_datetime`

Objetivo: Resolver o erro anterior substituindo `'1900/01/01'` por `NaN` (`np.nan`), repetir a tentativa de conversão com `pd.to_datetime()` e imprimir o conjunto de dados.

> **Nota Pedagógica:** Ao repetir o `pd.to_datetime()` , a linha 22 passa a ser ignorada como nula, mas o comando esbarra no segundo erro previsto no Passo 10: a string `'20201226'` (na linha 26) não corresponde ao formato `'%Y/%m/%d'` .

In [10]:
```python
# Passo 9a: Substituição do valor '1900/01/01' por NaN
df_tratado['Date'] = df_tratado['Date'].replace('1900/01/01', np.nan)

# Passo 9b: Repetição do passo de transformação para datetime (capturando o erro es
print('--- Segunda tentativa de conversão com pd.to_datetime (Passo 9b) ---')
try:
    df_tratado['Date'] = pd.to_datetime(df_tratado['Date'])
except ValueError as e:
    print(f'ValueError esperado (Passo 10): {e}')

# Passo 9c: Impressão do conjunto de dados
print('\n--- Estado do DataFrame após o Passo 9 ---')
print(df_tratado.to_string())
```

```
--- Segunda tentativa de conversão com pd.to_datetime (Passo 9b) ---
ValueError esperado (Passo 10): time data "20201226" doesn't match format "'%Y/%m/%
d'". You might want to try:
    - passing `format` if your strings have a consistent format;
    - passing `format='ISO8601'` if your strings are all ISO8601 but not necessarily
in exactly the same format;
    - passing `format='mixed'`, and the format will be inferred for each element ind
ividually. You might want to use `dayfirst` alongside this.

--- Estado do DataFrame após o Passo 9 ---
    ID  Duration          Date  Pulse  Maxpulse  Calories
0    0        60  '2020/12/01'    110       130    4091.0
1    1        60  '2020/12/02'    117       145    4790.0
2    2        60  '2020/12/03'    103       135    3400.0
3    3        45  '2020/12/04'    109       175    2824.0
4    4        45  '2020/12/05'    117       148    4060.0
5    5        60  '2020/12/06'    102       127    3000.0
6    6        60  '2020/12/07'    110       136    3740.0
7    7       450  '2020/12/08'    104       134    2533.0
8    8        30  '2020/12/09'    109       133    1951.0
9    9        60  '2020/12/10'     98       124    2690.0
10  10        60  '2020/12/11'    103       147    3293.0
11  11        60  '2020/12/12'    100       120    2507.0
12  12        60  '2020/12/12'    100       120    2507.0
13  13        60  '2020/12/13'    106       128    3453.0
14  14        60  '2020/12/14'    104       132    3793.0
15  15        60  '2020/12/15'     98       123    2750.0
16  16        60  '2020/12/16'     98       120    2152.0
17  17        60  '2020/12/17'    100       120    3000.0
18  18        45  '2020/12/18'     90       112       0.0
19  19        60  '2020/12/19'    103       123    3230.0
20  20        45  '2020/12/20'     97       125    2430.0
21  21        60  '2020/12/21'    108       131    3642.0
22  22        45           NaN    100       119    2820.0
23  23        60  '2020/12/23'    130       101    3000.0
24  24        45  '2020/12/24'    105       132    2460.0
25  25        60  '2020/12/25'    102       126    3345.0
26  26        60      20201226    100       120    2500.0
27  27        60  '2020/12/27'     92       118    2410.0
28  28        60  '2020/12/28'    103       132       0.0
29  29        60  '2020/12/29'    100       132    2800.0
30  30        60  '2020/12/30'    102       129    3803.0
31  31        60  '2020/12/31'     92       115    2430.0
```

Passos 10 e 11: Correção do Registro `'20201226'` e Conversão Completa de `Date` para `datetime`

Objetivo:

1. **Passo 10:** Transformar especificamente o valor `'20201226'` (linha 26) combinando os métodos `.replace()` e `pd.to_datetime('20201226', format='%Y%m%d')`.
2. **Passo 11:** Executar novamente `pd.to_datetime()` sobre toda a coluna `Date` e imprimir o conjunto de dados atualizado.

In [11]:
```
# Passo 10: Transformação específica do valor '20201226' combinando replace e to_da
df_tratado['Date'] = df_tratado['Date'].replace(
    '20201226',
    pd.to_datetime('20201226', format='%Y%m%d')
)

# Passo 11: Transformação de todos os dados da coluna 'Date' para o formato datetim
df_tratado['Date'] = pd.to_datetime(df_tratado['Date'])

# Impressão para verificar que todas as datas foram convertidas (e a linha 22 torno
print(df_tratado.to_string())
```

```
    ID  Duration       Date  Pulse  Maxpulse  Calories
0    0        60 2020-12-01    110       130    4091.0
1    1        60 2020-12-02    117       145    4790.0
2    2        60 2020-12-03    103       135    3400.0
3    3        45 2020-12-04    109       175    2824.0
4    4        45 2020-12-05    117       148    4060.0
5    5        60 2020-12-06    102       127    3000.0
6    6        60 2020-12-07    110       136    3740.0
7    7       450 2020-12-08    104       134    2533.0
8    8        30 2020-12-09    109       133    1951.0
9    9        60 2020-12-10     98       124    2690.0
10  10        60 2020-12-11    103       147    3293.0
11  11        60 2020-12-12    100       120    2507.0
12  12        60 2020-12-12    100       120    2507.0
13  13        60 2020-12-13    106       128    3453.0
14  14        60 2020-12-14    104       132    3793.0
15  15        60 2020-12-15     98       123    2750.0
16  16        60 2020-12-16     98       120    2152.0
17  17        60 2020-12-17    100       120    3000.0
18  18        45 2020-12-18     90       112       0.0
19  19        60 2020-12-19    103       123    3230.0
20  20        45 2020-12-20     97       125    2430.0
21  21        60 2020-12-21    108       131    3642.0
22  22        45        NaT    100       119    2820.0
23  23        60 2020-12-23    130       101    3000.0
24  24        45 2020-12-24    105       132    2460.0
25  25        60 2020-12-25    102       126    3345.0
26  26        60 2020-12-26    100       120    2500.0
27  27        60 2020-12-27     92       118    2410.0
28  28        60 2020-12-28    103       132       0.0
29  29        60 2020-12-29    100       132    2800.0
30  30        60 2020-12-30    102       129    3803.0
31  31        60 2020-12-31     92       115    2430.0
```

Passos 12 e 13: Remoção de Registros Nulos e Verificação Final

Objetivo:

1. **Passo 12:** Remover os registros contendo valores nulos (`.dropna(subset=['Date'])`), eliminando a linha 22 (`NaT`).
2. **Passo 13:** Imprimir o DataFrame final tratado (31 registros completos) e confirmar a estrutura final com `.info()` .

In [12]:

```python
# Passo 12: Remoção dos registros contendo valores nulos com base na coluna 'Date'
df_tratado = df_tratado.dropna(subset=['Date'])

# Passo 13: Impressão do DataFrame final e confirmação das transformações
print('--- Passo 13: DataFrame Final Tratado (31 Registros) ---')
print(df_tratado.to_string())

print('\n--- Informações Gerais do DataFrame Final ---')
df_tratado.info()
```

```
--- Passo 13: DataFrame Final Tratado (31 Registros) ---
    ID  Duration       Date  Pulse  Maxpulse  Calories
0    0        60 2020-12-01    110       130    4091.0
1    1        60 2020-12-02    117       145    4790.0
2    2        60 2020-12-03    103       135    3400.0
3    3        45 2020-12-04    109       175    2824.0
4    4        45 2020-12-05    117       148    4060.0
5    5        60 2020-12-06    102       127    3000.0
6    6        60 2020-12-07    110       136    3740.0
7    7       450 2020-12-08    104       134    2533.0
8    8        30 2020-12-09    109       133    1951.0
9    9        60 2020-12-10     98       124    2690.0
10  10        60 2020-12-11    103       147    3293.0
11  11        60 2020-12-12    100       120    2507.0
12  12        60 2020-12-12    100       120    2507.0
13  13        60 2020-12-13    106       128    3453.0
14  14        60 2020-12-14    104       132    3793.0
15  15        60 2020-12-15     98       123    2750.0
16  16        60 2020-12-16     98       120    2152.0
17  17        60 2020-12-17    100       120    3000.0
18  18        45 2020-12-18     90       112       0.0
19  19        60 2020-12-19    103       123    3230.0
20  20        45 2020-12-20     97       125    2430.0
21  21        60 2020-12-21    108       131    3642.0
23  23        60 2020-12-23    130       101    3000.0
24  24        45 2020-12-24    105       132    2460.0
25  25        60 2020-12-25    102       126    3345.0
26  26        60 2020-12-26    100       120    2500.0
27  27        60 2020-12-27     92       118    2410.0
28  28        60 2020-12-28    103       132       0.0
29  29        60 2020-12-29    100       132    2800.0
30  30        60 2020-12-30    102       129    3803.0
31  31        60 2020-12-31     92       115    2430.0

--- Informações Gerais do DataFrame Final ---
<class 'pandas.DataFrame'>
Index: 31 entries, 0 to 31
Data columns (total 6 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   ID        31 non-null     int64
 1   Duration  31 non-null     int64
 2   Date      31 non-null     datetime64[us]
 3   Pulse     31 non-null     int64
 4   Maxpulse  31 non-null     int64
 5   Calories  31 non-null     float64
dtypes: datetime64[us](1), float64(1), int64(4)
memory usage: 1.7 KB
```